CP – Projeto 1: NYC Taxi com Spark

Pedro Carmo (77904) & Rodrigo Carretas (77935) – novembro 2026

1 Introduction

Usámos os NYC TLC Yellow Taxi Trip Records de janeiro a setembro de 2018–2020: 27 ficheiros Parquet (2,3 GB, 161 M viagens; 154,7 M após limpeza) e a tabela de 265 zonas. Com a pandemia, o volume mensal varia mais de 30× (134 MB em mar/2018, 4 MB em abr/2020). Tarefas: **M1** limpeza, com contagem por ano das viagens removidas por critério; **M2** procura por *borough*, dia da semana e hora; **M3** estatísticas mensais por zona, com *ranking*, quota rolante de 3 meses e anomalias de volume; e as opcionais **O1** (anomalias de tarifa/km) e **O2** (fluxos origem-destino).

Motivação. O schema muda entre anos, o que obriga a ler e converter cada ficheiro separadamente; os ficheiros são desiguais e têm um único *row group*; as cinco operações reutilizam o mesmo dataset, o que expõe o custo da avaliação *lazy*; e há *joins*, agregações sobre 10^8 linhas e *windows* num cluster pequeno (6 cores, ∼2 GB de memória unificada) onde os dados não cabem em cache.

2 Pipeline

Etapas. (1) Ingestão: cada ficheiro é lido, convertido para um schema fixo e unido com `unionByName`; (2) limpeza: 7 critérios de validade removem 4% das viagens; (3) operações M1–O2, com *join* à tabela de zonas e escrita em Parquet. **Decisões.** Usámos a DataFrame API: os critérios de limpeza são expressões construídas programaticamente e reutilizadas para filtrar e contar (M1), o que em SQL textual seria frágil; as duas interfaces geram o mesmo plano no Catalyst. No M3, um índice contínuo de mês (12 ano + mês) faz o `rangeBetween` respeitar meses em falta. **Plano físico do M2** (linhas por etapa): `Scan parquet` (154,7 M) → `BroadcastHashJoin` com zonas (sem *shuffle*) → `HashAggregate` parcial (∼20 mil) → `Exchange` (1,4 MB) → `HashAggregate` final (1 003) → `WriteFiles`. O Catalyst escolhe sozinho o *broadcast* e a agregação parcial, pelo que só 1,4 MB atravessam a rede.

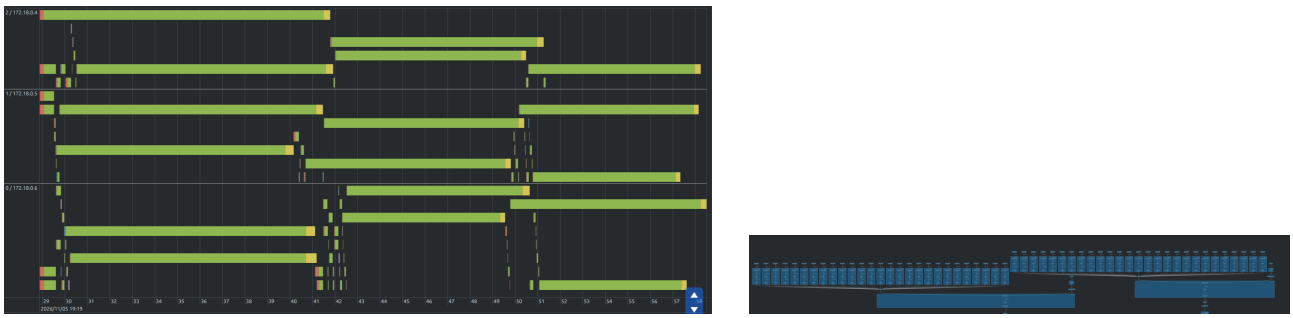
3 Profiling and Baseline

Setup. Portátil (*CPU/RAM: Intel® Core™ Ultra 7 268V × 8/32 GiB*), Spark 4.2.0 *standalone* em Docker: 3 workers × 2 cores × 2 GB, `executor.memory`=1,5 GB (∼740 MB de memória unificada por executor), AQE ativo e 200 partições de *shuffle* (valores por defeito). Tempos medidos no driver (mediana de 3 runs); métricas da Spark UI e da API REST.

	M1	M2	M3	O1	O2	Total
Baseline (s)	25,1	30,5	58,7	68,0	70,7	252,9

Table 1: Tempo do baseline por operação (mediana de 3 runs, dataset completo).

(B1) Releituras. Cada run lê e limpa os 161 M registos **9 vezes** (∼90% do tempo): o M3 lê 322 M registos (o mesmo agregado em dois ramos, Fig. 1b) e o O2 lê 484 M (o mesmo DataFrame recalculado em 3 ações). **(B2) Leitura desequilibrada.** Cada ficheiro é partido em ∼7 *splits*, mas só um recebe o *row group*: mais de 75% das tasks leem 0 registos (Fig. 1a). O impacto é pequeno: a eficiência do stage é de $102/(6 \times 18{,}2) \approx 93\%$. **(B3) Shuffle no O1.** Sem estatísticas do agregado (6 195 linhas), o Catalyst planeia um `SortMergeJoin` e faz *shuffle* das 154,7 M linhas (1,6 GB); o AQE só converte para *broadcast* depois de esse *shuffle* estar escrito. Não houve *spill*, UDFs Python nem *skew* de chave.



(a) (b)

Figure 1: Baseline: (a) timeline da leitura do M2, com tasks longas (uma por ficheiro) e muitas vazias; (b) DAG do M3, com o mesmo agregado calculado em dois ramos. Pedimos desculpa por a imagem não estar melhor mas foi a melhor versão que conseguimos

4 Optimisation Study

As otimizações foram aplicadas de forma cumulativa, uma de cada vez, e medidas por operação. Os tempos por operação não incluem a escrita do dataset limpo, que é reportada à parte.

4.1 Materialização do dataset limpo

(a) B1 e B2: cada operação relia os 27 ficheiros e repetia casts, *union* e filtragem (18–28 s por leitura). **(b)** Como as leituras são limitadas por CPU (CPU ≈ 88% do tempo das tasks), ler um dataset já limpo e tipado deve ser bem mais barato. **(c)** Escrevemos uma vez o `trips_clean` (8 colunas). Testámos três variantes: `repartition(24)`, `repartitionByRange(24, pickup)` e sem repartição, com `outputTimestampType=TIMESTAMP_MICROS` (o Spark escreve INT96 por defeito). **(d)** Cada leitura passou para 5–10 s. A repartição equilibrou as tasks (6,2 M registos em todas), mas custa caro: o *round-robin* baralha as linhas e degrada a compressão (3,4 GB, 4,9 GB de *spill*, escrita de ∼78 s); o *range* mantém a ordem (2,6 GB) mas exige uma passagem de amostragem e um *shuffle* de 4,6 GB (76–127 s). Sem repartição a escrita desce para ∼**29 s** (2,4 GB; runs isoladas). Como a eficiência da leitura já era de 93% (B2), o equilíbrio perfeito só a levava a 96%, o que não compensa o *shuffle*. Ficámos com a variante sem repartição.

4.2 `persist` com `coalesce` (M3 e O2)

(a) B1: o M3 calculava o mesmo agregado em dois ramos e o O2 recalculava o mesmo DataFrame (`od`, 659 mil linhas) em três ações. **(b)** Guardar estes resultados pequenos em memória elimina as releituras. **(c)** `persist()` do agregado do M3 e do `od`. **(d)** M3: de 23,7 para apenas 21,7 s. O `persist` removeu o segundo scan, mas o plano em cache mantém as 200 partições do *shuffle* (o AQE não pode juntá-las num plano em cache), e as windows seguintes passaram a correr 4 stages de 200 tasks sobre 7 mil linhas. Com `coalesce(1)` antes do `persist`, o M3 desceu para 17,6 s. O2: com `coalesce(6)` (uma partição por core), de ∼29 para 11,6 s, com um só scan e leituras da cache em 6 tasks.

4.3 Agregar antes do *join*

(a) O M3 fazia o *join* com as zonas sobre as 154,7 M linhas e agregava por (*borough*, zona, ano, mês), com strings na chave. **(b)** Agregar primeiro por `PULocationID` (inteiro) e juntar as zonas sobre o resultado reduz o *join* a milhares de linhas e simplifica a chave. **(c)** Reordenámos M2, M3 e O1. **(d)** O efeito depende da cardinalidade da pré-agregação. **M3**: o mesmo número de grupos (cada ID tem uma zona), agregação de 1,3 min para 40 s de CPU, e o M3 de 17,6 para ∼10 s. **O1**: os grupos passam de ∼84 para ∼2 600 por task, o que dá um ganho marginal (∼3 s). **M2**: agrupar por zona em vez de *borough* cria 37× mais grupos (686 mil linhas parciais em vez de 22 mil) e a agregação piorou de 1,5 para 2,1 min de CPU, pelo que mantivemos a ordem original.

4.4 *Broadcast* explícito (O1)

(a) B3: *shuffle* de 1,6 GB por um `SortMergeJoin` planeado sem saber que o agregado tinha 6 195 linhas. **(b)** Indicar ao Catalyst que a tabela é pequena evita o *shuffle* logo no planeamento. **(c)** `F.broadcast(bucket_stats)`. **(d)** O plano inicial passa a `BroadcastHashJoin`, o *shuffle* desce de 1,6 GB para ∼1 MB e o O1 passa de ∼32 para 19 s (3 stages pesados para 2).

4.5 Resumo

	M1*	M2	M3	O1	O2	Total
Baseline (s)	20,6	27,5	59,2	68,4	71,2	253,3
Final (s)	68,4	10,1	7,8	13,8	9,3	109,6
Speedup	0,3×	2,7×	7,6×	5,0×	7,7×	**2,3**×

Table 2: Baseline vs versão final (mediana de 4 e 3 runs, respetivamente). *Na versão final o M1 inclui a escrita do `trips_clean`, amortizada pelas operações seguintes. Contributos: materialização (todas as leituras, 18–28 → 5–10 s); `persist`+`coalesce` (M3, O2); agregar antes do *join* (M3; marginal no O1; rejeitado no M2); *broadcast* explícito (O1).

5 Analysis

Escala do dataset. Com 10, 25, 50 e 100% dos registos, a versão final demorou 58,6, 68,0, 81,2 e 112,2 s (Fig. 2a): 10× menos dados só a tornam 1,9× mais rápida. A causa é o M1 com a escrita (42–70 s), porque a amostragem é feita depois da leitura e os 2,3 GB originais são sempre lidos por inteiro; só a escrita diminui. As operações M2–O2 escalam de forma aproximadamente linear acima de 25% (20,7, 28,7 e 43,2 s, cerca de +14 s por cada 50%), com um custo fixo de ∼12 s (planeamento, lançamento de stages e abertura de ficheiros) que domina a 10% (15,4 s). **Tamanho do cluster.** Com 1, 2 e 3 workers (2, 4 e 6 cores), a versão final demorou 152,8, 117,7 e 109,6 s, um *speedup* de apenas 1,30× e 1,39×, longe do ideal de 3× (Fig. 2b). O tempo de CPU dos executores mostra porquê: para o mesmo trabalho, o M1 consome 157, 218 e 262 s de CPU com 2, 4 e 6 cores (M2–O2: 104, 139 e 161 s). Cada core rende menos quando há mais, porque os workers são contentores no mesmo portátil e partilham os cores físicos, a memória e o disco [confirmar o nº de cores físicos com `lscpu`]. Acrescentar workers não acrescenta capacidade real de cálculo.

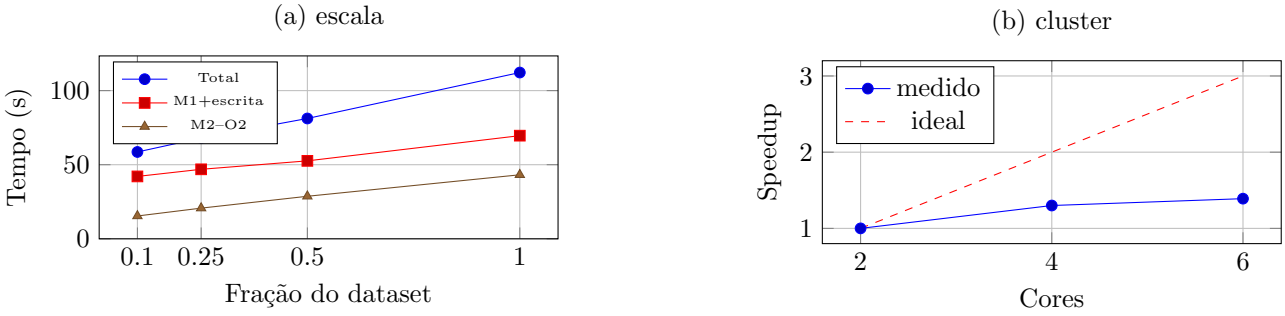


Figure 2: Versão final: (a) tempo vs fração do dataset; (b) *speedup* vs número de cores.

6 Discussion

Bottlenecks por resolver. A escrita do `trips_clean` e o M1 leem os dados originais em duas passagens separadas (∼68 s no total, 62% do tempo da versão final); juntá-las exigiria *cache* do dataset bruto, que não cabe em memória. As restantes operações são dominadas pela agregação sobre 10^8 linhas, já perto do limite de CPU do cluster. **Limitações.** O cluster corre em contentores num único portátil, partilhando CPU e disco com o sistema; medimos variações de ±10% entre runs idênticas (o M1 do baseline variou entre 15 e 25 s), pelo que diferenças pequenas não são conclusivas. Os tempos também variam entre sessões (baseline: 215–237 s numa sessão e 233–268 s noutra; versão final: 108–112 s, ou 124–130 s com partilha de ecrã ativa), pelo que comparamos medianas de várias runs. Várias versões intermédias foram medidas com poucas repetições, e as otimizações foram avaliadas de forma cumulativa, o que torna o efeito de cada uma dependente das anteriores: a materialização compensava com 9 leituras, mas quase deixou de compensar quando o `persist` as reduziu para 5. **Com mais tempo**, mediríamos cada otimização isoladamente sobre o baseline, testaríamos num cluster real (onde a repartição e o equilíbrio de tasks ganham peso), reescreveríamos os ficheiros de origem com *row groups* mais pequenos.

Acknowledgements e utilização de IA

Utilizámos ferramentas de inteligência artificial generativa em duas fases do trabalho. **(1) Construção da baseline:** a implementação inicial das operações M1–O2 (`baseline.ipynb`) foi gerada com apoio de IA (Claude) a partir do enunciado, sendo depois revista e corrigida por nós; foi deliberadamente mantida sem otimizações

(sem *cache*, sem *broadcast* explícito e com as configurações por defeito) para servir de ponto de partida. **(2) Especificações técnicas:** durante o estudo de otimização, usámos um assistente de IA (Claude, da Anthropic) para esclarecer conceitos do Spark (por exemplo, a divisão de ficheiros Parquet em *splits* e *row groups*, a interação do AQE com `persist`, as estratégias de *join* e o formato `INT96`), para rever o texto deste relatório e também para gerar o makefile. As hipóteses foram testadas por nós: todas as medições foram feitas no nosso cluster, e as decisões e conclusões são da nossa responsabilidade.